Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
(ФГАОУ ВО СПбПУ)
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

КУРСОВАЯ РАБОТА

По дисциплине «Верстка и прототипирование сайтов»

Разработка игрового приложения «Матрешка» (семестр 5)

Студент
группы з5130903/10301 М.В. Федоров

Оценка выполненной студентом работы:
Преподаватель,
старший преподаватель Е.В. Комарова

Санкт-Петербург – 2024

САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ПЕТРА ВЕЛИКОГО
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

ЗАДАНИЕ

на выполнение курсовой работы по дисциплине

«Верстка и прототипирование сайтов»

Федорову Михаилу Валерьевичу группа: з5130903/10301
(фамилия, имя, отчество) (номер группы)
семестр: 5

1. Тема работы: Разработка игрового приложения Матрешки

2. Срок сдачи студентом законченной работы: 23.12.2023

3. Исходные данные по работе: «Верстка и прототипирование сайтов».

4. Содержание работы (перечень подлежащих разработке вопросов):
4.1. Введение

4.2. Тип игры.

4.3. Структура приложения.

4.4. Описание работы приложения
4.5. Код приложения.

4. Дата выдачи задания: «18» ноября 2023г

5. Перечень графического материала (с указанием обязательных рисунков): схематизация структуры приложения и порядка выполнения (диаграммы классов UML, диаграммы IDEF0).
6. Дата выдачи задания: «18» ноября 2023 г.

Преподаватель _____ Е.В. Комарова
(подпись) (инициалы, фамилия)

Студент _____ М.В. Федоров
(подпись) (инициалы, фамилия)

Содержание

Введение..... 3

1. Проектирование.....5

2. Структура игры.....6

3. Дизайн игры..... 7

4. Логика игры..... 11

Заключение.....15

Список использованной литературы..... 16

Приложение А...... 17

Введение

В процессе курсовой работы мы будем поэтапно создавать игру под названием "Матрешки". Эта игра включает в себя сборку матрешек из трех частей разного размера и цвета. Мы будем выбирать необходимые технологии, проектировать интерфейс, разрабатывать логику игры и проводить тестирование. Мы рассмотрим все этапы разработки, начиная с концепции и дизайна игры, и заканчивая тестированием. После завершения разработки игры мы исследуем возможности улучшения игрового процесса и дизайна.

Таким образом, целью курсовой работы является полное ознакомление с процессом разработки игры "Матрешки", включая концепцию, логику и дизайн, а также рассмотрение возможностей для улучшения игры.

1. Проектирование

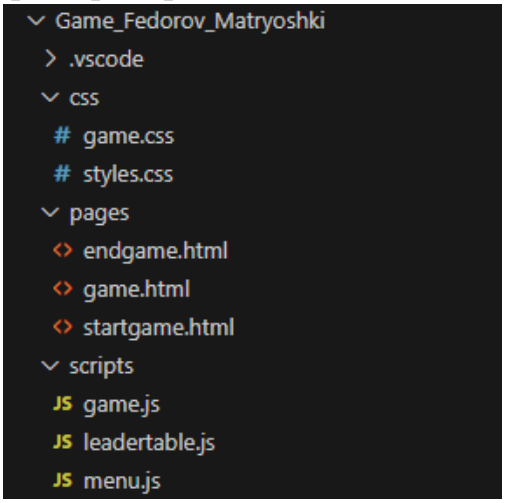
Для начала создания игры необходимо определить набор технологий, на основе которых будет разработана вся логика и дизайн. В процессе выполнения курсовой работы мы будем использовать HTML, CSS и JavaScript. Концепция игры включает несколько уровней различной сложности, прохождение которых позволяет игроку зарабатывать очки. По завершении игры игрок сможет просмотреть таблицу лидеров и сравнить свои результаты с другими участниками. При начале игры игроку будет предложено ввести своё имя и выбрать уровень сложности. Чем сложнее уровень, тем больше очков получает игрок за одну собранную матрешку. Во время игры идет отсчет таймера, по истечении времени которого, заканчивается игра. За каждую собранную матрешку к этому таймеру прибавляется немного времени. Задача игрока собрать как можно больше матрешек.

На рисунке 1 представлено окно с описанием сложности.

Рисунок 1. Окно с описанием сложности игры.

2. Структура игры

Структура игры представляет собой набор файлов формата .html, .css, .js.



Файл startgame.html представляет собой начальную страницу игры, где происходит ввод имени игрока и выбор сложности игры.

В дальнейшем, после старта игры, происходит переход непосредственно к странице игры game.html, где происходит вся логика игры и подсчёт результатов игрока. После окончания игры происходит переход на страницу endgame.html, где игрока уведомляют о количестве набранных им очков и показывается таблица лидеров.

Представим, также, графическое представление игры:

Рисунок 2. Графическое представление игры

3. Дизайн игры

Давайте рассмотрим проектирование дизайна игры, начиная с главной страницы. Для общего дизайна главной страницы и экрана завершения игры (за исключением игрового окна) будет использован минималистичный тёмный стиль. Основным цветом фона страницы будет темно-серый, а цветом текста - белый. На протяжении всей игры в верхнем углу будет расположена кнопка для перехода на главную страницу сайта или на страницу начала игры. Также на главной странице есть описание различий каждого уровня сложности.

Пример дизайна главной страницы игры:

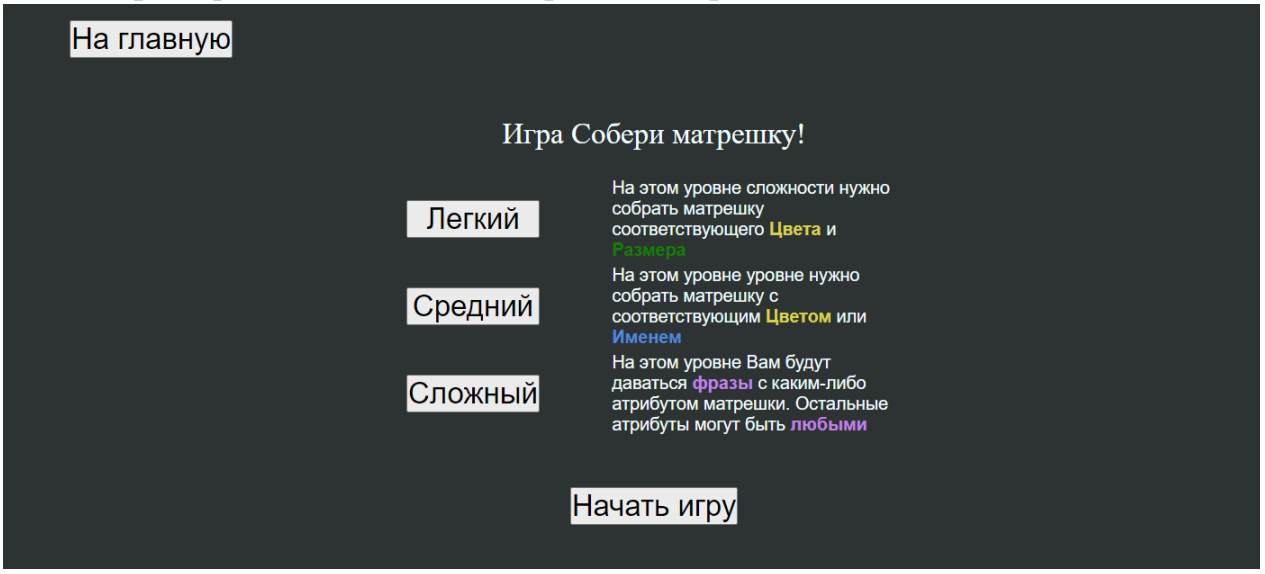


Рисунок 3. Пример дизайна главной страницы игры

В дизайне самой игры используется противоположный подход: фон цвета бежевый и черный текст.

Игра подразумевает собирание нужных матрешек. Поэтому игроку нужно показать какую матрешку ему нужно собрать и что он уже собрал.

Из-за этой особенности окно игры разделено на три части: слева вверху показан пример матрешки, а внизу то, что игрок уже собрал, справа – основное окно игры, где происходит выбор нужной матрешки. Матрешки имеют различный цвет, размер и имя.

Посмотрим на дизайн окна игры:

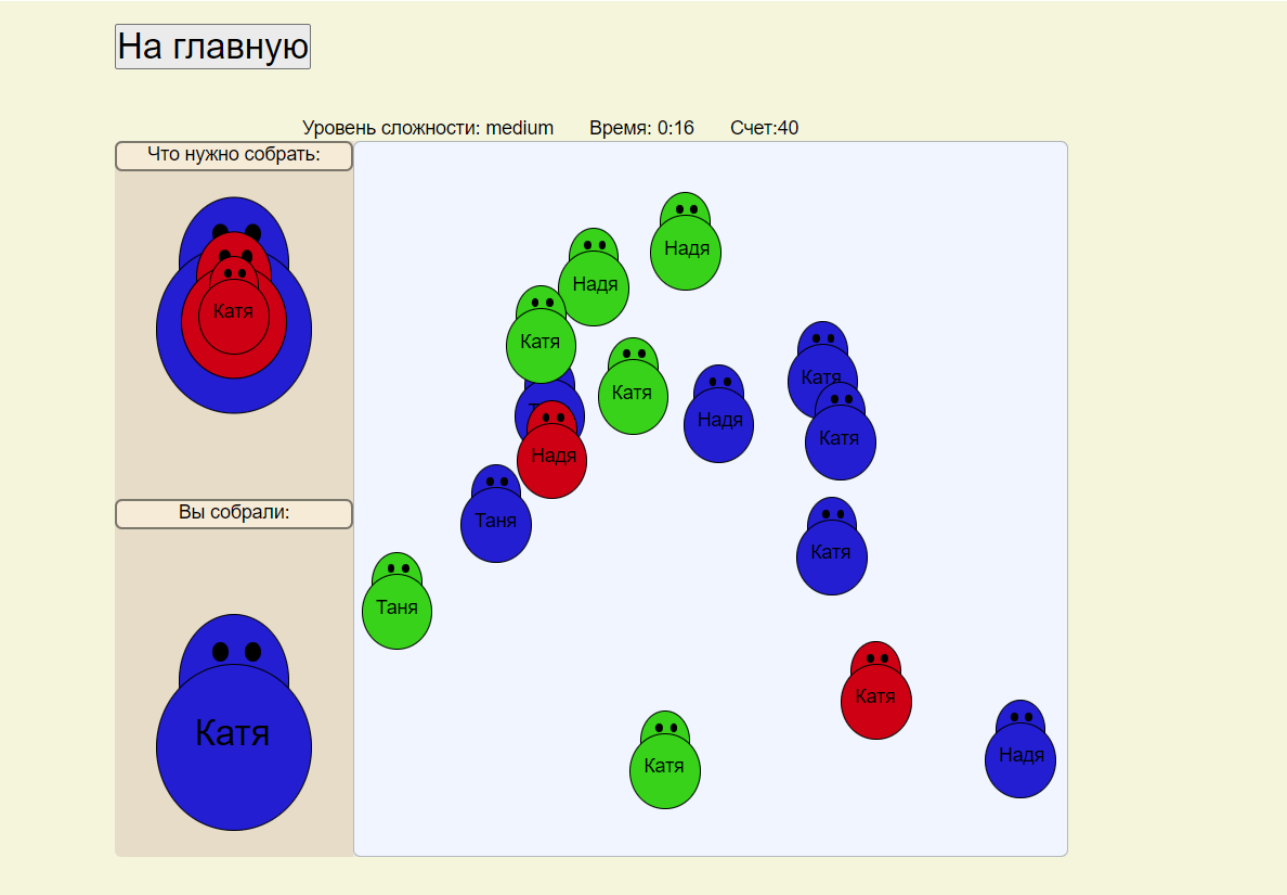


Рисунок 4. Дизайн окна игры

Вернемся к дизайну главной страницы и рассмотрим HTML элементы, связанные с этой страницей. Главная страница игры содержит блок . Внутри этого блока находятся теги с приветственным заголовком, кнопкой начала игры и текстовым описанием уровней.

```
<div id="welcome">
    <p> Игра Собери матрешку!</p>
    <div id = "blocks">
        <div class="block">
            <button class="btn">Легкий</button>
            <button class="btn">Средний</button>
            <button class="btn">Сложный</button>
        </div>
        <div class="block">
            <div class="desc">На этом  уровне сложности нужно собрать матрешку
            <div class="desc">На этом уровне уровне нужно собрать матрешку с со
            <div class="desc">На этом уровне нужно собрать матрешку  с соответс
        </div>
    </div>
    <div class="startbtn">
        <button class="btn" id="start" onclick="menu()">Начать игру</button>
    </div>
</div>
```

Рисунок 5. HTML-код начальной страницы игры

При нажатии игры вызывается JavaScript функция menu(), осуществляющая ввод имени игрока и переход к окну выбора сложности.

В Приложении А представлен листинг полного кода JavaScript данной и других страниц. Окно выбора сложности, завершения игры и таблицы лидеров разработаны аналогичным образом.

Например, страница завершения игры будет представлена следующим HTML-кодом:

```
<div id="welcome">
    <p id = "score"></p>
    <div id = "leaders">Таблица лидеров:</div>
    <div id = "tab"></div>
    <div class="startbtn">
        <button class="btn" id="start" onclick="menu()">Еще раз!</button>
    </div>
</div>
```

Рисунок 6. HTML-код страницы завершения игры

Посмотрим на HTML-код страницы игры:

```
<section class="game">
    <div class="wrapper">
        <a href="startgame.html" ><button class="btn">На главную</button></a>
        <div class="game_wrap">
            <div class = "indicator">
                <p id = "level">Уровень сложности: </p>
                <p id = "time">Время: 0:00</p>
                <p id = "scope">Счет: 0</p>
            </div>
            <div class="menu">
                <div id="example">
                    <div class="heading">
                        <p>Что нужно собрать:</p>
                    </div>
                </div>
                <div id="assembly">
                    <div class="heading">
                        <p>Вы собрали:</p>
                    </div>
                </div>
            </div>
            <div id="field"></div>
        </div>
    </div>
</section>
```

Рисунок 7. HTML-код страницы игры

Пример дизайна страницы окончания игры:

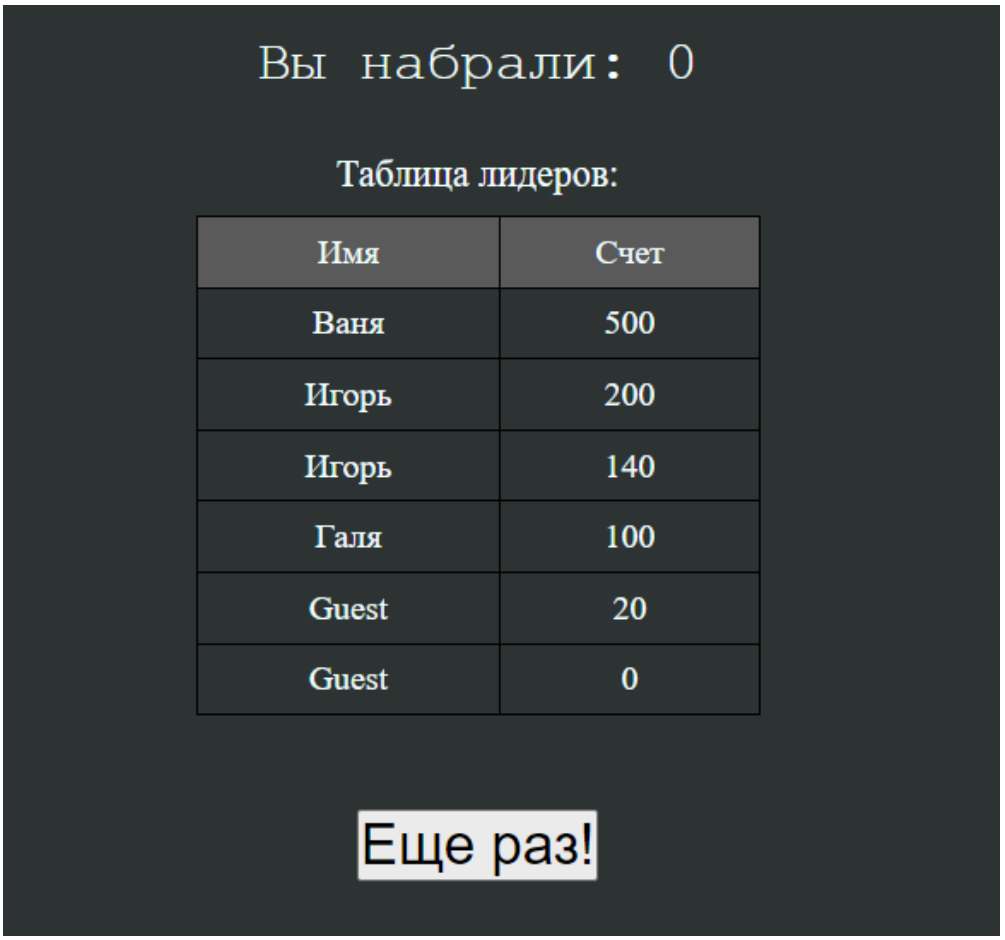


Рисунок 8. Пример дизайна страницы окончания игры

После оформления дизайна, переходим к разработке логики игры.

4. Логика игры

Процесс игры представляет собой следующее окно:

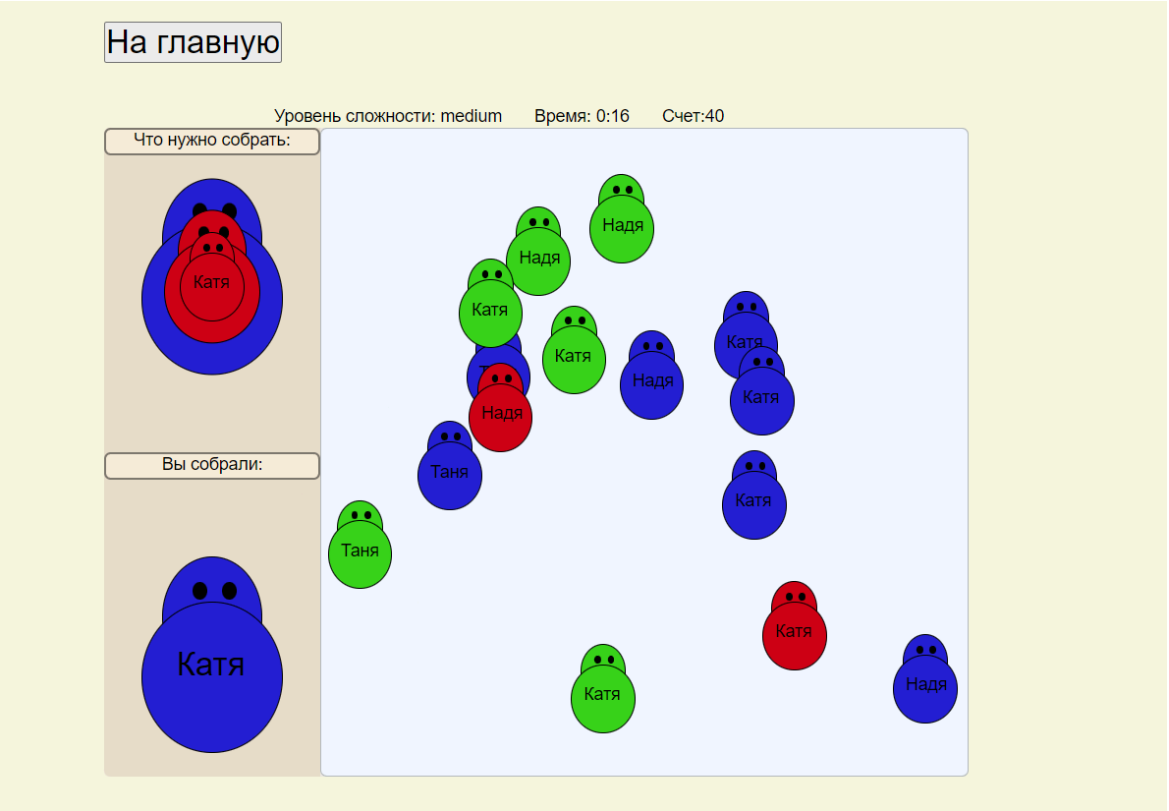


Рисунок 9. Окно игры

Цель игрока: последовательно собирать матрешку, указанную в примере. Изначально все матрешки маленького размера. Увеличить или уменьшить матрешку на поле можно при помощи нажатия левой кнопки мыши. собирать матрешки можно путем нажатия на нее левой кнопкой мыши или перетаскивая ее в поле “Вы собрали”. Матрешки нужно собирать с самой большой и заканчивать маленькой. Всего матрешка может быть трех размеров: большая, средняя, маленькая.

Изначально игроку дается время, по истечении которого игра заканчивается. За каждую собранную матрешку, игроку начисляются очки и добавляется немного времени.

На лёгком уровне сложности случайным образом появляются матрешки различной окраски. Матрешки собираются путем левого клика. На этом уровне сложности при успешном собирании матрешки игроку дается 100

очков и дополнительные 5 секунд.

Пример игры на легком уровне сложности:

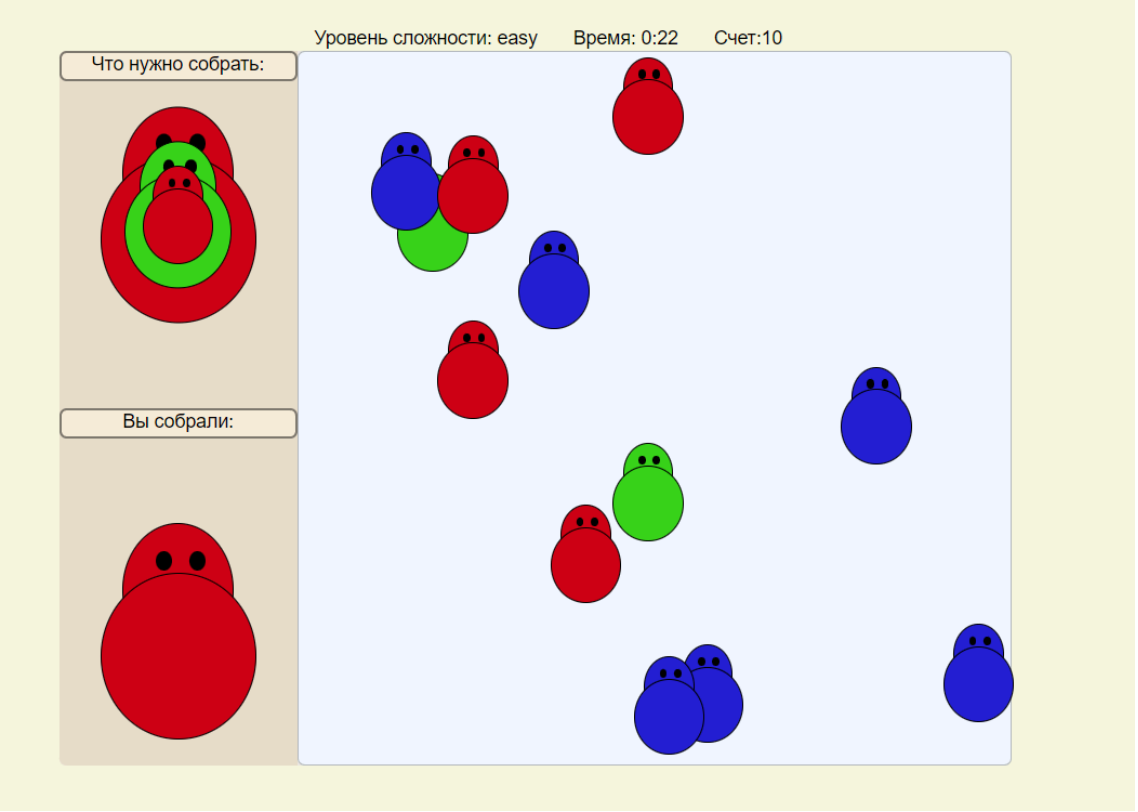


Рисунок 10. Пример игры на легком уровне сложности

На среднем уровне у матрешек появляются имена. Теперь нужно собрать матрешку либо с нужным цветом, либо с нужным именем. При этом матрешки также как и раньше собираются по размеру. На этом уровне сложности матрешки нужно самому помещать в поле “Вы собрали” путем нажатия и удержания левой кнопки мыши и перемещения курсора в поле. При успешном собирании матрешки игроку дается уже 220 очков и дополнительные 10 секунд.

Пример игры на среднем уровне сложности:

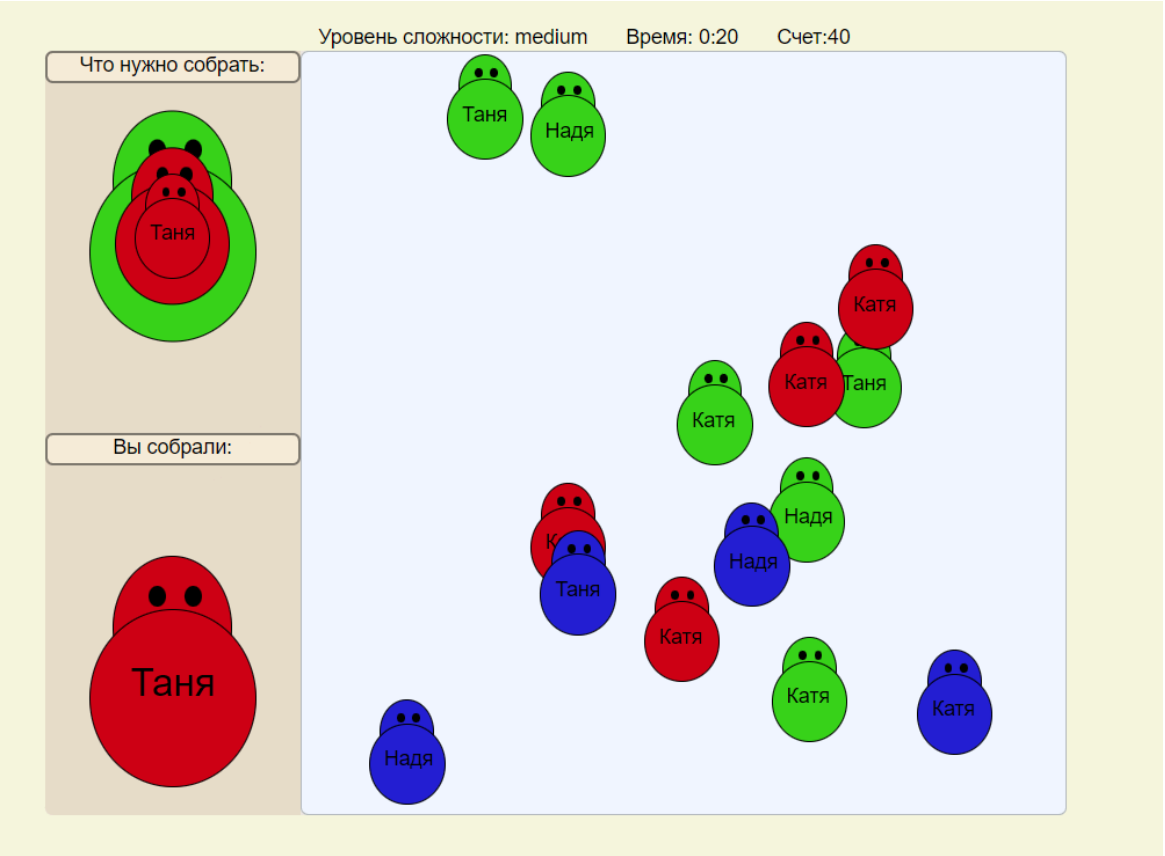


Рисунок 11. Пример игры на среднем уровне сложности

На сложном уровне у матрешек появляется анимация движения по кругу. Игроку показывается текст с описанием того, что ему нужно собрать. Матрешки, как и на среднем уровне, нужно самому перемещать в поле “Вы собрали”. При успешном собирании матрешки игроку дается целых 400 очков и дополнительно 20 секунд.

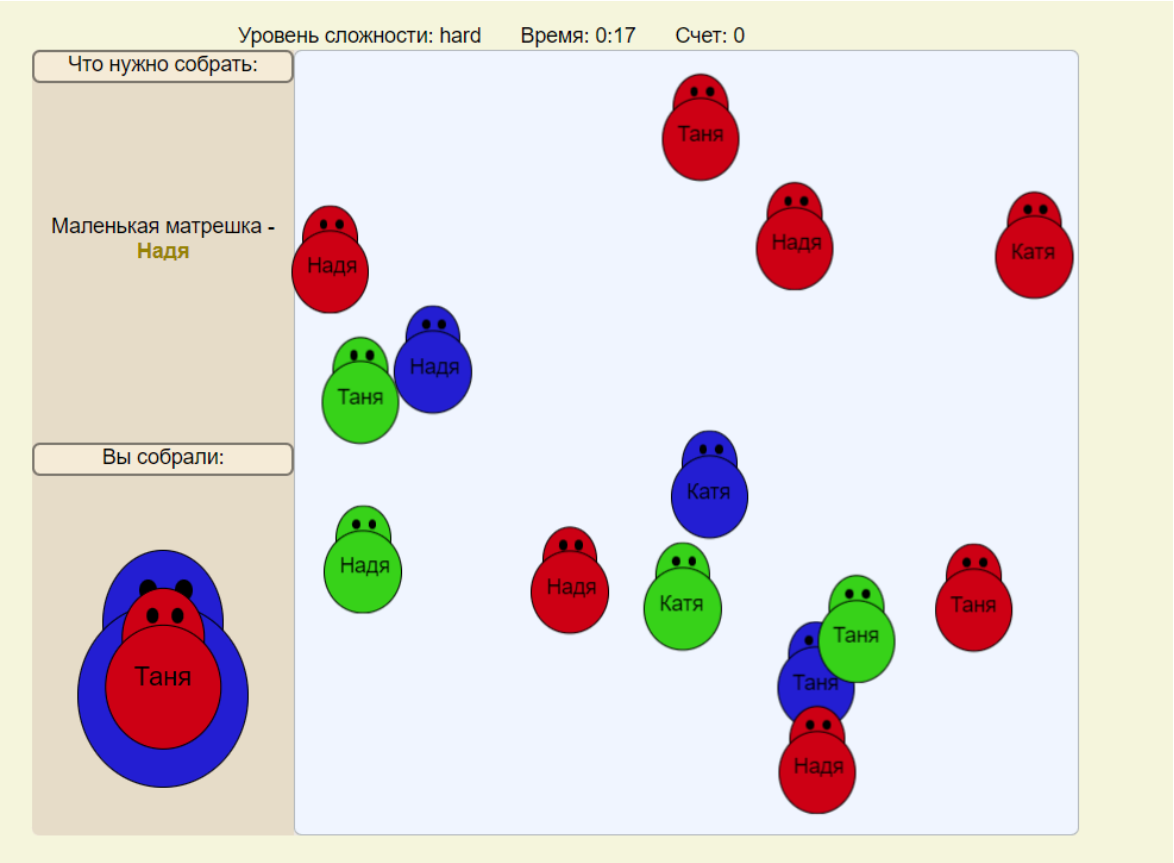


Рисунок 12. Пример игры на сложном уровне

После истечения времени, выводится сообщение “Время истекло!” и игрока переносит на страницу окончания игры. Где отображаются набранные им очки и таблица лидеров. Игроку предоставляется возможность сыграть еще раз. Тогда нужно заново ввести имя и выбрать уровень сложности.

Посмотрим на таблицу лидеров:

Таблица лидеров:

Имя	Счет
Ваня	500
Игорь	200
Игорь	140
Галя	100
Guest	40
Guest	20
Guest	10
Guest	0

Рисунок 13. Таблица лидеров

Таблица лидеров разработана таким образом, чтобы отображать десятку лучших игроков. Благодаря использованию объекта localStorage в JavaScript, результаты предыдущих игроков сохраняются, и их можно просматривать даже после закрытия браузера и повторного открытия.

Внутри JavaScript кода, таблица лидеров представлена массивом объектов, имеющих два поля: имя и очки. При завершении игры, код сохраняет результаты текущего игрока в данный массив, после чего сортирует массив и превращает в JSON-строку, которая отправляется в localStorage. Таким образом, при получении из localStorage JSON-строки мы восстанавливаем массив и используем его для построения окна таблицы лидеров.

Заключение

В ходе написания данной курсовой работы мы разработали приложение-игру «Матрешка», представляющей собой процесс построения матрешки начиная с самой большой и заканчивая маленькой.

Разрабатывая игру, мы использовали современные технологии веб-разработки HTML, CSS и JavaScript. При помощи данных технологий была реализована логика игры и дизайн. Благодаря возможностям данных технологий была реализована анимация и визуальные эффекты для создания удобного и понятного пользовательского интерфейса.

Список использованной литературы

1. HTML5 Book: https://html5book.ru/;
2. Методическое пособие по курсу: https://dl.spbstu.ru/mod/resource/view.php?id=157007;
3. Современный учебник JavaScript: https://learn.javascript.ru;
4. JavaScript. Подробное руководство. – пер. с англ. – СПБ: Символ-Плюс, 2008;
5. Алгоритмы. – пер. с англ. – Москва: Издательство МЦНМО, 2014.

Приложение А.

В данном приложении представлен полный код приложения-игры.

Файл startgame.html:

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Магазин цветов</title>
    <link rel="stylesheet" type="text/css" href="../css/styles.css">
</head>
<body>
    <main class="main">
        <section class="startgame">
            <div class="wrapper">
                <a href="../../index.html" ><button class="btn">На
главную</button></a>
            <div id="welcome">
                <p> Игра Собери матрешку!</p>
                <div id = "blocks">
                    <div class="block">
                        <button class="btn">Легкий</button>
                        <button class="btn">Средний</button>
                        <button class="btn">Сложный</button>
                    </div>
                    <div class="block">
                        <div class="desc">На этом  уровне сложности нужно собрать
матрешку соответствующего <span style="color: rgb(221, 211,
72);"><strong>Цвета</strong></span> и <span style="color:
green;"><strong>Размера</strong></span></div>
                        <div class="desc">На этом уровне уровне нужно собрать
матрешку с соответствующим <span style="color: rgb(221, 211,
72);"><strong>Цветом</strong></span> или <span style="color: rgb(72, 137,
221);"><strong>Именем</strong></span></div>
                        <div class="desc">На этом уровне Вам будут даваться <span
style="color: rgb(194, 134, 234);"><strong>фразы</strong></span> с каким-либо
атрибутом матрешки. Остальные атрибуты могут быть <span style="color: rgb(194, 134,
234);"><strong>любыми</strong></span></div>
                    </div>
                </div>
                <div class="startbtn">
```

```
                    <button class="btn" id="start" onclick="menu()">Начать
игру</button>
                </div>
            </div>
            <div id="chooseDifficulty">
                <p>Выберите сложность:</p>
                <div>
                    <button class="btn" id="easy">Легкая</button>
                    <button class="btn" id="medium">Средняя</button>
                    <button class="btn" id="hard">Сложная</button>
                </div>
            </div>
        </section>
        <script src="../scripts/menu.js"></script>
    </main>
</body>
</html>
```

Файл game.html:

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Магазин цветов</title>
    <link rel="stylesheet" type="text/css" href="../css/styles.css">
    <link rel="stylesheet" type="text/css" href="../css/game.css">
</head>
<body>
    <main class="main">
        <section class="game">
            <div class="wrapper">
                <a href="startgame.html" ><button class="btn">На
главную</button></a>
                <div class="game_wrap">
                    <div class = "indicator">
                        <p id = "level">Уровень сложности: </p>
                        <p id = "time">Время: 0:00</p>
                        <p id = "scope">Счет: 0</p>
                    </div>
                    <div class="menu">
                        <div id="example">
                            <div class="heading">
```

```
                        <p>Что нужно собрать:</p>
                    </div>
                </div>
                <div id="assembly">
                    <div class="heading">
                        <p>Вы собрали:</p>
                        <p id = "win"></p>
                    </div>
                </div>
            </div>
            <div id="field"></div>
        </div>
    </div>
  </section>
 <script src="../scripts/game.js"></script>
</main>
</body>
</html>
```

Файл endgame.html:

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Магазин цветов</title>
    <link rel="stylesheet" type="text/css" href="../css/styles.css">
</head>
<body>
    <main class="main">
        <section class="startgame">
            <div class="wrapper">
                <a href="startgame.html" ><button class="btn">На
главную</button></a>
            <div id="welcome">
                <p id = "score"></p>
                <div id = "leaders">Таблица лидеров:</div>
                <div id = "tab"></div>
                <div class="startbtn">
                    <button class="btn" id="start" onclick="menu()">Еще
раз!</button>
                </div>
            </div>
```

```html
            <div id="chooseDifficulty">
                <p>Выберите сложность:</p>
                <div>
                    <button class="btn" id="easy">Легкая</button>
                    <button class="btn" id="medium">Средняя</button>
                    <button class="btn" id="hard">Сложная</button>
                </div>
            </div>
        </section>
        <script src="../scripts/menu.js"></script>
        <script src="../scripts/leadertable.js"></script>
    </main>
</body>
</html>
```

Файл styles.css:

```css
*{
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

.wrapper{
    max-width: 1200px;
    margin: 0 auto;
    width: 100%;
    padding: 0 10px;
}

.startgame {
    background: rgb(49, 51, 51);
    height: 100vh;
    color: azure;
    font-size: large;
}
.btn {
    margin-top: 50px;
    font-size: 30px;
}

#welcome {
    display: flex;
    flex-direction: column;
```

```
    justify-content: center;
    align-items: center;
    margin-top: 60px;
}
#welcome p, #chooseDifficulty p{
    font-size: 30px;
}
#blocks{
    width: 500px;
    display: flex;
    flex-direction:row;
    justify-content: space-around;
}
.block {
    display: flex;
    flex-direction:column;
    justify-content: space-around;
    margin-top: 25px;
    font-family: Arial, Helvetica, sans-serif;
}
.block .btn{
    margin-top: 25px;
    margin-bottom: 25px;
}

.desc {
    margin-left: 20%;
}
#chooseDifficulty{
    display: none;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    margin-top: 150px;
}
#chooseDifficulty div {
    display: flex;
    flex-direction: column;
}

#score {
    margin-bottom: 30px;
    font-family: 'Courier New', Courier, monospace;
```

```
}
#leaders {
    font-size: 20px;
}

#tab table {
    width: 100px;
    border-collapse: collapse;
    margin-top: 10px;
    width: 300px;
}

th, td {
    border: 1px solid black;
    padding: 8px;
    text-align: center;
}
```

Файл game.css:

```
.game_wrap {
        font-family: Arial, Helvetica, sans-serif;
        background-color: rgb(238, 234, 229);
        margin-top: 60px;
        margin-bottom: 100px;
        display: flex;
        max-width: 800px;
        height: 600px;
        border-radius: 6px;
        flex-direction:row;
        justify-content: center;
        resize: none;
    }
    .game{
    background:beige;
        height: 100vh;
    }
    .indicator {
        display: flex;
        position: absolute;
        flex-direction:row;
        justify-content:space-between;
        margin-top: -20px;
        left: 25%;
```

```
}
.indicator p {
    margin-right: 30px;
}

#field {
    position: relative;
    background-color: aliceblue;
    width: 600px;
    border: 1px solid rgba(128, 128, 128, 0.5);
    border-radius: 6px;
}

.menu{
    width: 200px;
    height: 100%;
    border-radius: 6px;
}
.heading {
    width: 100%;
    height: 25px;
    border: 2px solid rgba(10, 10, 10, 0.5);
    background-color: antiquewhite;
    border-radius: 6px;
}
.heading p {
    text-align: center;
}
#example {
    display: flex;
    flex-direction: column;
    align-items: center;
    width: 100%;
    height: 50%;
    background-color: rgb(235, 221, 204);
    border-radius: 6px 0px 0px;
}
#example .replic {
    text-align: center;
    margin-top: 100px;
    width: 180px;
}
```

```
#example .matryoshka {
    margin-top: 100px;
}
#assembly {
    display: flex;
    justify-content: center;
    position: relative;
    width: 100%;
    height: 50%;
    background-color: rgb(235, 221, 204);
    border-radius: 0px 0px 0px 6px;
}

#win{
   margin-top: 20px;
   font-size: 30px;
   display: none;
}

.matryoshka {
    width: 50px;
    height: 75px;
    position: absolute;
}


.matryoshka .body {
    background-color: #d46012;
    width: 119%;
    height: 85%;
    top:  20%;
    left: -9.5%;
    border-radius: 50%;
    position: absolute;
    pointer-events: none;
    z-index: 1;
    border: 1px solid black;
}

.matryoshka .head {
    background-color: #d46012;
    width: 85%;
    height: 68%;
```

```
    border-radius: 50%;
    position: absolute;
    top:  -5%;
    left: 7.5%;
    pointer-events: none;
    z-index: 0;
    border: 1px solid black;
}

.matryoshka .head .eyes {
    background-color: black;
    width: 15%;
    height: 15%;
    border-radius: 50%;
    position: absolute;
    top: 20%;
    left: 30%;
    pointer-events: none;
}
.matryoshka p {
    text-align: center;
    position: absolute;

    top: 45%;
    left: 14%;
    pointer-events: none;
    z-index: 2;
}


.green .body {
    background-color: #22d412;
}

.green .head {
    background-color: #22d412;
}

.red .body {
    background-color: #d41212;
}

.red .head {
```

```
    background-color: #d41212;
}


.blue .body {
    background-color: #121fd4;
}

.blue .head {
    background-color: #121fd4;
}

.small {
    z-index: 3;
}
.medium {
    width: 75px;
    height: 112.5px;
    z-index: 2;
}
.big {
    width: 110px;
    height: 165px;
    z-index: 1;
}

.medium p{
    font-size: 20px;
    left: 20%;
}

.big p{
    font-size: 30px;
    left: 20%;
}

.animLine {
    animation: moveInLine 1s linear infinite;
}
.animCircle{
    animation: moveInCircle 1.5s linear infinite;
}
@keyframes moveInLine{
```

```
        0% {
            transform: translateX(0%);
        }
        25% {
            transform: translateX(-25%);
        }
        50% {
            transform: translateX(0%);
        }
        75% {
            transform: translateX(25%);
        }
        100% {
            transform: translateX(0%);
        }
    }

    @keyframes moveInCircle{
        0% {
            transform: translateY(15%);
        }
        25% {
            transform: translateX(15%);
        }
        50% {
            transform: translateY(-15%);
        }
        75% {
            transform: translateX(-15%);
        }
        100% {
            transform: translateY(15%);
        }
    }
```

Файл menu.js:

```
function menu() {
        const welcome = document.getElementById('welcome');
        welcome.style.display = 'none';
        const chooseDifficulty = document.getElementById('chooseDifficulty');
        chooseDifficulty.style.display = 'flex';
        const easyButton = document.getElementById('easy');
        const mediumButton = document.getElementById('medium');
```

```
    const hardButton = document.getElementById('hard');

    function clickHandler(difficulty) {
        chooseDifficulty.style.display = 'none';
        localStorage.setItem('difficulty', difficulty);
        location.href = 'game.html';
    }

    easyButton.addEventListener('click', () => clickHandler('easy'));

    mediumButton.addEventListener('click', () => clickHandler('medium'));

    hardButton.addEventListener('click', () => clickHandler('hard'));

    let name = prompt("Введите свое имя (пустая строка или отмена = Гость):");
    if (name === null || name === "") {
        name = 'Guest';
    } else if (name.length > 10) {
        name = name.slice(0, 10);
    }
    localStorage.setItem('name', name);
}
```

Файл leadertable.js:

```
let scoreText = document.getElementById("score");
    const localStorageScore = localStorage.getItem('score');
    scoreText.textContent = "Вы набрали:  " + localStorageScore;

    const localStorageData = localStorage.getItem('myArray');
    const localStorageName = localStorage.getItem('name');
    let myArray = JSON.parse(localStorage.getItem('myArray')) || [];

    if (localStorageName !== null || localStorageScore !== null) {
        myArray.push({name:localStorageName, score: localStorageScore});
    }
    myArray.sort((a, b) => b.score - a.score);
    myArray = myArray.slice(0, 9);
    localStorage.removeItem('name');
    localStorage.removeItem('score');

    localStorage.setItem('myArray', JSON.stringify(myArray));

    leaderboard();
```

```
function leaderboard(){

const newTable = document.createElement('table');
const headerRow = newTable.insertRow();
const headerName = headerRow.insertCell();
headerName.textContent = 'Имя';
headerName.style.backgroundColor = 'rgb(92, 91, 91)';
const headerScore = headerRow.insertCell();
headerScore.textContent = 'Счет';
headerScore.style.backgroundColor = 'rgb(92, 91, 91)';

myArray.forEach(player => {
    const row = newTable.insertRow();
    const nameCell = row.insertCell();
    nameCell.textContent = player.name;
    const scoreCell = row.insertCell();
    scoreCell.textContent = player.score;
});

document.getElementById("tab").appendChild(newTable);
}
```

Файл game.js:

```
const difficulty = localStorage.getItem('difficulty') || 'easy';
const field = document.getElementById('field');
var field_minx = field.getBoundingClientRect().x;
var field_maxx = field.getBoundingClientRect().right;
var field_miny = field.getBoundingClientRect().y;
var field_maxy = field.getBoundingClientRect().bottom;
const assemblyContainer = document.getElementById('assembly');
const exampleContainer = document.getElementById('example');
const scope =  document.getElementById('scope');
const time =  document.getElementById('time');
const level = document.getElementById('level');

const green = 'green';
const red = 'red';
const blue = 'blue';
const small = 'small';
const medium = 'medium';
const big = 'big';

let numReplic;
```

```
        let arrayName = [];
        let arrayOnPlace = [];

        let minutes = 0;
        let seconds = 30;
        let count = 0;
        let color;
        let name;
        let arrayMatryoshka = [];

        level.textContent = 'Уровень сложности: ' + difficulty;

        if (difficulty !== "easy") {
                arrayName = ['Таня', 'Надя', 'Катя'];
        }

        const n = 15;
        const m = [];
        for (var i=0; i<5; i++) {
            m.push(addElem(getRandomName()));
        }

        if (difficulty === "hard") {
            addReplic();
        } else {addElemExmaple(getRandomMatryoshka());}

        generate();

        function eventClick(event) {
            if (event.target.classList.contains(arrayMatryoshka[0].size) &&
event.target.classList.contains(arrayMatryoshka[0].color)) {
                arrayMatryoshka.shift();
                const elemClone = event.target.cloneNode(true);
                moveElementToAssembly(elemClone);
                event.target.remove();
                count +=10;
                scope.textContent = 'Счет:' + count;
                if (arrayMatryoshka.length === 0) {
                    count +=100;
                    scope.textContent = 'Счет:' + count;
                    winText(true);
                    addTime(5);
                    update();
```

```
            }
        }
    }


    function mouseDown(e) {
        e.target.classList.remove('animCircle');
        const elem = e.target;
        const elemArea = elem.parentElement;

        const { clientX: mouseLeft, clientY: mouseTop } = e;
        const { left: elemLeft, top: elemTop } = elem.getBoundingClientRect();
        const { left: areaLeft, top: areaTop } =
elemArea.getBoundingClientRect();

        const shiftX = mouseLeft - elemLeft + areaLeft;
        const shiftY = mouseTop - elemTop + areaTop;

        previusZIndex = elem.style.zIndex;
        elem.style.zIndex = 20;
        elem.style.cursor = 'grabbing';

        function moveAt(x, y) {
            elem.style.left = x + 'px';
            elem.style.top = y + 'px';
        }

        function mouseMove(e) {
            moveAt(e.pageX - shiftX, e.pageY - shiftY);
        }

        function mouseUp() {
            elem.style.zIndex = previusZIndex;
            elem.style.cursor = 'grab';
            elem.removeEventListener('mouseup', mouseUp);
            document.removeEventListener('mousemove', mouseMove);
            if (difficulty === "hard") {
                elem.classList.add('animCircle');
                checkHard(elem, assemblyContainer)
            } else {checkMedium(elem, assemblyContainer)}
        }
        elem.addEventListener('mouseup', mouseUp);
        document.addEventListener('mousemove', mouseMove);
```

```
        }

        function checkMedium(elem, assemblyContainer) {
            if (collision(elem, assemblyContainer) &&
elem.classList.contains(arrayMatryoshka[0].size) &&
(elem.classList.contains(arrayMatryoshka[0].color) ||
elem.classList.contains(arrayMatryoshka[0].name))){
                arrayMatryoshka.shift();
                const elemClone = elem.cloneNode(true);
                moveElementToAssembly(elemClone);
                elem.remove();
                count +=40;
                scope.textContent = 'Счет:' + count;
                if (arrayMatryoshka.length === 0) {
                    count +=220;
                    scope.textContent = 'Счет:' + count;
                    winText(true);
                    addTime(10);
                    update();
                }
            }
        }

        function checkHard(elem, assemblyContainer) {
            console.log(arrayOnPlace[0]);
            if (collision(elem, assemblyContainer)) {
                if (elem.classList.contains('big') && arrayOnPlace[0] === undefined)
{
                    elem.classList.remove('animCircle');
                    const elemClone = elem.cloneNode(true);
                    moveElementToAssembly(elemClone);
                    elem.remove();
                    arrayOnPlace[0] = elemClone;
                }
                if (elem.classList.contains('medium') && arrayOnPlace[0] !==
undefined && arrayOnPlace[1] === undefined) {
                    elem.classList.remove('animCircle');
                    const elemClone = elem.cloneNode(true);
                    moveElementToAssembly(elemClone);
                    elem.remove();
                    arrayOnPlace[1] = elemClone;
                }
                if (elem.classList.contains('small') && arrayOnPlace[1] !== undefined
```

```
&& arrayOnPlace[2] === undefined) {
                    elem.classList.remove('animCircle');
                    const elemClone = elem.cloneNode(true);
                    moveElementToAssembly(elemClone);
                    elem.remove();
                    arrayOnPlace[2] = elemClone;
                }
                let count = 0;
                arrayOnPlace.forEach(element => {
                    if(element !== undefined) count++;
                });
                if (count === 3) {
                    switch (numReplic) {
                        case 0:
                            if (arrayOnPlace[0].classList.contains("red")) {
                                count +=400;
                                scope.textContent = 'Счет:' + count;
                                winText(true);
                                addTime(20);
                                updateHard();
                            } else {winText(false); updateHard();}
                        break;
                        case 1:
                            if (arrayOnPlace[1].classList.contains("green")) {
                                count +=400;
                                scope.textContent = 'Счет:' + count;
                                winText(true);
                                addTime(20);
                                updateHard();
                            } else {winText(false); updateHard();}
                        break;
                        case 2:
                            if (arrayOnPlace[0].classList.contains("blue")) {
                                count +=400;
                                scope.textContent = 'Счет:' + count;
                                winText(true);
                                addTime(20);
                                updateHard();
                            } else {winText(false); updateHard();};
                        break;
                        case 3:
                            if (arrayOnPlace[2].classList.contains("Надя")) {
                                count +=400;
```

```
                                scope.textContent = 'Счет:' + count;
                                winText(true);
                                addTime(20);
                                updateHard();
                            } else {winText(false); updateHard();}
                        break;
                        case 4:
                            if (arrayOnPlace[1].classList.contains("Катя")) {
                                count +=400;
                                scope.textContent = 'Счет:' + count;
                                winText(true);
                                addTime(20);
                                updateHard();
                            } else {winText(false); updateHard();}
                        break;
                    }
                }
            }
        }

        function collision(elem, assemblyContainer) {
            const { left: ringLeft, top: ringTop } = elem.getBoundingClientRect();
            const { left: blockLeft, top: blockTop } =
assemblyContainer.getBoundingClientRect();

            return ringLeft >= blockLeft && ringLeft <= blockLeft +
assemblyContainer.offsetWidth && ringTop >= blockTop && ringTop <= blockTop +
assemblyContainer.offsetHeight;
        }

        function update() {
            arrayMatryoshka = getRandomMatryoshka();
            let elementsToAssembly =
assemblyContainer.querySelectorAll('.matryoshka');
            setTimeout(() => {
                elementsToAssembly.forEach(element => {
                    element.remove();
                });
            }, 1000);
            let elementsToExample = exampleContainer.querySelectorAll('.matryoshka');
            elementsToExample.forEach(element => {
                element.remove();
            });
```

```
            addElemExmaple(arrayMatryoshka);
        }

        function updateHard() {
            let elementsToAssembly =
assemblyContainer.querySelectorAll('.matryoshka');
            setTimeout(() => {
                elementsToAssembly.forEach(element => {
                    element.remove();
                });
            }, 1000);
            let elementToExample = exampleContainer.querySelector('.replic');
            elementToExample.remove();
            arrayOnPlace.splice(0, arrayOnPlace.length);
            addReplic();
        }

        function eventRightClick(event) {
            event.preventDefault();

            if (event.target.classList.contains(small)) {
                event.target.classList.remove(small);
                event.target.classList.add(medium);
                return;
            }

            if (event.target.classList.contains(medium)) {
                event.target.classList.remove(medium);
                event.target.classList.add(big);
                return;
            }

            if (event.target.classList.contains(big)) {
                event.target.classList.remove(big);
                event.target.classList.add(small);
                return;
            }
        }

        function addElem(name) {
            color = getRandomColor();
            name = getRandomName();
            let fieldContainer = document.getElementById('field');
```

```
    let matryoshkaDiv = document.createElement('div');
    matryoshkaDiv.classList.add('matryoshka', color, small, name);

    let bodyDiv = document.createElement('div');
    bodyDiv.classList.add('body');

    let headDiv = document.createElement('div');
    headDiv.classList.add('head');

    let eyesDiv1 = document.createElement('div');
    eyesDiv1.classList.add('eyes');

    let eyesDiv2 = document.createElement('div');
    eyesDiv2.classList.add('eyes');
    eyesDiv2.style.left = '60%';

    let nameParagraph = document.createElement('p');
    nameParagraph.textContent = name;

    headDiv.appendChild(eyesDiv1);
    headDiv.appendChild(eyesDiv2);

    matryoshkaDiv.appendChild(bodyDiv);
    matryoshkaDiv.appendChild(headDiv);
    matryoshkaDiv.appendChild(nameParagraph);
    matryoshkaDiv.addEventListener('mouseover', function() {
        matryoshkaDiv.style.cursor = 'pointer';
    });
    if (difficulty === "hard") {matryoshkaDiv.classList.add('animCircle');}
    if (difficulty === "easy") {
        matryoshkaDiv.addEventListener('click', eventClick);
    } else {
        matryoshkaDiv.addEventListener('mousedown', mouseDown);
    }
    matryoshkaDiv.addEventListener('contextmenu', eventRightClick);
    fieldContainer.appendChild(matryoshkaDiv);
    setPosition(matryoshkaDiv);
    return matryoshkaDiv;
}

function setPosition(matryoshkaDiv){
    matryoshkaDiv.style.top = Math.floor((field_maxy - field_miny -
```

```
matryoshkaDiv.getBoundingClientRect().height)*Math.random()) + 'px';
            matryoshkaDiv.style.left = Math.floor((field_maxx - field_minx -
matryoshkaDiv.getBoundingClientRect().width)*Math.random()) + 'px';
        }

        function addElemExmaple(arrayMatryoshka) {
            for (i = 0; i < 3; i++) {
                createElemExmaple(arrayMatryoshka[i].size, arrayMatryoshka[i].color,
arrayMatryoshka[i].name);
            }
        }

        function addReplic() {
            let replic = document.createElement('p');
            let span = document.createElement('span');
            let textNode;
            numReplic = Math.floor(Math.random() * 5);
            switch (numReplic) {
                case 0:
                    span = document.createElement('span');
                    span.textContent = 'красная';
                    span.style.color = 'red';
                    span.style.fontWeight = 'bold';
                    textNode = document.createTextNode('Большая матрешка - ');
                    replic.appendChild(textNode);
                    replic.appendChild(span);
                    replic.classList.add('replic');
                    exampleContainer.appendChild(replic);
                    break;
                case 1:
                    span = document.createElement('span');
                    span.textContent = 'зеленая';
                    span.style.color = 'green';
                    span.style.fontWeight = 'bold';
                    textNode = document.createTextNode('Средняя матрешка - ');
                    replic.appendChild(textNode);
                    replic.appendChild(span);
                    replic.classList.add('replic');
                    exampleContainer.appendChild(replic);
                    break;
                case 2:
                    span = document.createElement('span');
                    span.textContent = 'синяя';
```

```
            span.style.color = 'blue';
            span.style.fontWeight = 'bold';
            textNode = document.createTextNode('Большая матрешка - ');
            replic.appendChild(textNode);
            replic.appendChild(span);
            replic.classList.add('replic');
            exampleContainer.appendChild(replic);
            break;
        case 3:
            span = document.createElement('span');
            span.textContent = 'Надя';
            span.style.color = 'rgb(153, 130, 0)';
            span.style.fontWeight = 'bold';
            textNode = document.createTextNode('Маленькая матрешка - ');
            replic.appendChild(textNode);
            replic.appendChild(span);
            replic.classList.add('replic');
            exampleContainer.appendChild(replic);
            break;
        case 4:
            span = document.createElement('span');
            span.textContent = 'Катя';
            span.style.color = 'rgb(153, 130, 0)';
            span.style.fontWeight = 'bold';
            textNode = document.createTextNode('Средняя матрешка - ');
            replic.appendChild(textNode);
            replic.appendChild(span);
            replic.classList.add('replic');
            exampleContainer.appendChild(replic);
            break;
    }
}

function createElemExmaple(size, color, name) {

    let matryoshkaDiv = document.createElement('div');
    matryoshkaDiv.classList.add('matryoshka', color, size, name);

    let bodyDiv = document.createElement('div');
    bodyDiv.classList.add('body');

    let headDiv = document.createElement('div');
    headDiv.classList.add('head');
```

```
    let eyesDiv1 = document.createElement('div');
    eyesDiv1.classList.add('eyes');

    let eyesDiv2 = document.createElement('div');
    eyesDiv2.classList.add('eyes');
    eyesDiv2.style.left = '60%';

    let nameParagraph = document.createElement('p');
    if (size === "small") {
        nameParagraph.textContent = name;
    }

    if (size === "medium") {
        matryoshkaDiv.style.transform = 'translate(0px, -18.75px)';
    } else if (size === "big") {
        matryoshkaDiv.style.transform = 'translate(0px, -45px)';
    }

    headDiv.appendChild(eyesDiv1);
    headDiv.appendChild(eyesDiv2);

    matryoshkaDiv.appendChild(bodyDiv);
    matryoshkaDiv.appendChild(headDiv);
    matryoshkaDiv.appendChild(nameParagraph);

    exampleContainer.appendChild(matryoshkaDiv);
}

function moveElementToAssembly(element) {
    let assemblyContainer = document.getElementById('assembly');
        assemblyContainer.appendChild(element);
        element.style.position = 'absolute';
        element.style.top = '30%';
        element.style.left = '22.5%';
        element.style.cursor = 'default';

        if (element.classList.contains('medium')) {
            element.style.transform = 'translate(17.5px, 26.25px)';
        } else if (element.classList.contains('small')) {
            element.style.transform = 'translate(30px, 45px)';
        }
```

```
}

function getRandomColor() {
    var num = Math.floor(Math.random() * 3) + 1;
    if (num === 1 ) {
        return green;
    }
    if (num === 2 ) {
        return red;
    }
    if (num === 3 ) {
        return blue;
    }
}

function getRandomMatryoshka() {
    let name = getRandomName(arrayName);
    arrayMatryoshka = [
        { size: 'big', color: getRandomColor(), name: name},
        { size: 'medium', color: getRandomColor(), name: name},
        { size: 'small', color: getRandomColor(), name: name}
    ];
    return arrayMatryoshka;
}

function getRandomName() {
    var num = Math.floor(Math.random() * arrayName.length);
    return arrayName[num];
}

function generate() {
    const i = setInterval(() => {
        const mo = addElem(getRandomColor());
        m.push(mo);
        if (m.length > n) {
            const k = m.shift();
            k.remove();
        }
    }, 1000)
}

function winText(flag) {
    const element = document.getElementById('win');
```

```
    element.style.display = 'block';
    if (flag) {
        element.textContent = 'Правильно!';
        element.style.color = 'green';
    } else {
        element.textContent = 'Неправильно!';
        element.style.color = 'red';
    }
    setTimeout(() => {
        element.style.display = 'none';
    }, 1000);
}

function updateTimer() {
    time.textContent = 'Время:' + ` ${minutes}:${seconds}`;
}

function addTime(sec) {
    seconds += sec;
    if (seconds > 60) {
        minutes += 1;
        seconds -= 60;
    }
    updateTimer();
}

let timerInterval = setInterval(function() {
    if (seconds > 0) {
        seconds -= 1;
    } else {
        if (minutes > 0) {
            minutes -= 1;
            seconds = 59;
        } else {
            clearInterval(timerInterval);
            alert ('Время истекло!');
            localStorage.setItem('score', count);;
            location.href = 'endgame.html';
        }
    }
    updateTimer();
}, 1000);
```